Updating the Power Sector Competitiveness Dashboard Simulator

When you need to add or remove indicators from your dashboard, follow these steps in indicator_formatting.R:

Step 1: Update the Base Data CSV

Edit state_indicator_data.csv:

- **Add columns** for new indicators with state-by-state values (0-1 scale)
- **Remove columns** for deprecated indicators

Step 2: Update indicator_steps

Define the slider step size for each indicator:

```
indicator_steps <- c(
  "Your New Indicator Name (C)" = 0.25,  # Add inside its category's block
  "Third Party PPA (C)" = 0.25,
  # ... existing indicators ...
  # Remove any deprecated indicators
)
```

Rules:

- Use (C) for Consumer, (S) for Structure, (M) for Regional Market
- Common step values: 0.2, 0.25, 0.5, 1.0
- Binary indicators (yes/no) → use 1.0
- A step of 0.33 creates a four-choice slider (0, 0.33, 0.66, 1). Every state's value for that indicator must be one of those four, or the app stops at startup.
- Indicators scored as weighted averages (for example, across a state's utilities or markets) use 0.01, so the slider can show the exact value.
- List indicators grouped by category, in the order Consumer, Structure, Regional Market. Put a new indicator inside its category's block.
- Each name must match the CSV column header exactly, including spaces and hyphens. Use the same name in indicator_info and indicator_scoring_details.

Step 3: Update indicator_groups_lookup

Adjust the counts in the rep() function to match your new totals:

```
indicator_groups_lookup <- data.frame(
  Indicator = names(indicator_steps),
  Group = c(
    rep("Consumer (C)", 7),       # ← Update count if you added/removed Consumer indicators
    rep("Structure (S)", 7),      # ← Update count if you added/removed Structure indicators
    rep("Regional Market (M)", 3)  # ← Update count if you added/removed Market indicators
  ),
  stringsAsFactors = FALSE
)
```

Important: The sum of these three numbers must equal length(indicator_steps).

Groups are assigned by position. This is why the order in Step 2 matters.

Step 4: Update indicator_info table

Add short descriptions for new indicators or remove old ones:

```
indicator_info <- list(
  "Your New Indicator Name (C)" =
    "Brief description of what this indicator measures and why it matters.",


  # ... existing indicators ...
)
```

Step 5: Update indicator_scoring_details table

Add scoring criteria for new indicators or remove old ones:

```
indicator_scoring_details <- list(
  "Your New Indicator Name (C)" =
    "1.0: Full description.<br>0.5: Partial description.<br>0.0: None.",
```

```
  # ... existing indicators ...
)
```

Note: Use
 for line breaks in the HTML table display.

The script stops with an error if indicator_steps, indicator_info, and indicator_scoring_details do not list the same names.

Step 6: Run the Script

Run indicator_formatting.R from the app folder to regenerate the revised RDS files:

- state_indicator_data.rds
- indicator_data.rds

The app reads these RDS files, not the CSV. Changes to the CSV take effect only after you rerun the script.

Step 7: Verify

The app needs the shinyWidgets package. On a new computer, install it first with install.packages("shinyWidgets").

The main app will now **automatically**:

- Create sliders for all indicators
- Update category counts in labels ("Consumer (C): (X indicators)")
- Update the effective weight distribution, which accounts for the number of indicators in each category
- Build detail tables with new descriptions
- Calculate scores correctly

The app also checks the data at startup. If something is wrong, it stops with a message naming the problem:

- **"...have no entry in indicator_groups_lookup":** a CSV column header does not match any name in indicator_formatting.R.
- **"...have no column in state_indicator_data":** a name in indicator_formatting.R has no matching CSV column.

- **"Missing or non-numeric values...":** a cell is blank or contains text. The message names the state and indicator.
- **"...0.33-step indicators have values other than 0, 0.33, 0.66, 1":** re-code the data, or change the step.

Spot-check a few states before publishing. With all weights at 100%, a state's score should equal the average of its row times 100. Switch to another state and back, and confirm the score does not change.

Step 8: Publish

- Click publish  to upload the changed application to the website
    - The files to publish are
        - scenario_simulator_PSCD.R
        - indicator_data.rds
        - plot_theming.R
        - state_indicator_data.rds

Quick Checklist

When adding an indicator:

- Add column to state_indicator_data.csv, with a header that matches the indicator name and a number in every cell
- Add entry to indicator_steps, inside its category's block
- Update count in indicator_groups_lookup
- Add entry to indicator_info
- Add entry to indicator_scoring_details
- Run the indicator_formatting.R script to generate RDS files
- Install shinyWidgets if this is a new computer
- Test in Shiny app
- Publish the application to Shiny

When removing an indicator:

- Remove column from state_indicator_data.csv
- Remove entry from indicator_steps
- Update count in indicator_groups_lookup
- Remove entry from indicator_info
- Remove entry from indicator_scoring_details

- Run script to generate RDS files
- Test in Shiny app

No changes needed in the main Shiny app code (scenario_simulator_PSCD.R) when adding or removing indicators within the three existing categories

All counts and the effective weight distribution update automatically. Adding, removing, or renaming a category requires changes to the app code: CATEGORY_NAMES, the weight sliders, and the tab CSS in scenario_simulator_PSCD.R, and COLOR_MAP in plot_theming.R.